Documento de Diseño (DI)

AuraBot: Robot Aspiradora Autónomo con Yocto

Jose Bernardo Barquero 2023150476
Alexander Montero 2023166058
Jimmy Feng 2023060347
Jose Edo Campos 2023135620

Instituto Tecnológico de Costa Rica
CE-1113 Sistemas Empotrados

14 de octubre de 2026

1. Introducción

AuraBot es un sistema embebido desarrollado para controlar un robot aspiradora autónomo basado en una Raspberry Pi 4. El proyecto integra navegación reactiva, control diferencial de motores, detección de obstáculos, reproducción de audio y una interfaz web para supervisar y controlar el robot de manera remota.

El sistema utiliza Linux construido mediante Yocto, con una imagen personalizada denominada `ce1113-p1`. El software se organiza en componentes independientes, de forma que el control del hardware, la interfaz web y la reproducción de audio puedan desarrollarse y mantenerse por separado.

El objetivo de este documento es justificar las principales decisiones de diseño de AuraBot y establecer cómo se comprobará el cumplimiento de sus requerimientos funcionales y de seguridad.

2. DI1: Identificación de necesidades y requerimientos

2.1. Necesidades identificadas

El problema de ingeniería consiste en integrar varios subsistemas en una plataforma embebida que debe operar con recursos limitados y responder a las condiciones del entorno físico.

En AuraBot se identifican las siguientes necesidades:

- Detectar obstáculos mediante sensores infrarrojos y reaccionar modificando el movimiento del robot.
- Controlar independientemente las velocidades y direcciones de dos motores DC.
- Permitir la selección entre operación autónoma y manual.
- Registrar el movimiento mediante encoders Hall y mantener un mapa incremental del área explorada.
- Reproducir música y sonidos de notificación sin depender de la interfaz gráfica de un sistema operativo convencional.
- Proporcionar una interfaz web con autenticación, telemetría y controles remotos.
- Construir una imagen Linux adaptada al proyecto y automatizar el inicio de sus servicios.

2.2. Configuración del hardware

La configuración definida en el código de AuraBot utiliza los siguientes pines GPIO de la Raspberry Pi 4:

Componente	**GPIO BCM**	**Función**
Motor izquierdo	8, 14 y 15	PWM y señales de dirección del puente H.
Motor derecho	7, 23 y 24	PWM y señales de dirección del puente H.
Sensor infrarrojo izquierdo	9	Detección de obstáculos.
Sensor infrarrojo derecho	10	Detección de obstáculos.
Encoder Hall izquierdo	5	Registro de pulsos del motor.
Encoder Hall derecho	6	Registro de pulsos del motor.
LED de servicio	22	Indicación del estado del sistema.
LED manual	17	Indicación del modo manual.
LED autónomo	27	Indicación del modo autónomo.
LED de obstáculo	4	Indicación de detección de obstáculos.

Cuadro 1: Asignación de pines definida en `hardware_config.h`.

Los sensores infrarrojos se configuran con lógica activa en bajo. El software interpreta las lecturas para representar de forma uniforme la presencia de obstáculos. Los encoders permiten estimar el movimiento de las ruedas y alimentar el cálculo de odometría.

2.3. Restricciones y seguridad

El sistema debe funcionar sobre una imagen Linux mínima, evitando paquetes innecesarios y manteniendo separados los componentes de software. El enunciado establece como referencias un sistema de archivos raíz de hasta 200 MB y un tiempo de arranque de hasta 15 segundos.

En cuanto a la seguridad eléctrica, los GPIO no deben alimentar directamente los motores. Se necesita una etapa de potencia adecuada y aislamiento entre la lógica y el circuito de los motores, según el diseño eléctrico seleccionado. La Raspberry Pi debe recibir una alimentación regulada de 5 V y corriente suficiente; la alimentación de los motores debe diseñarse de acuerdo con sus requerimientos eléctricos.

También deben considerarse los riesgos asociados con las baterías, los cortocircuitos y el movimiento inesperado durante las pruebas. Las primeras pruebas de motores deben realizarse con las ruedas levantadas y el robot asegurado.

3. DI2: Evaluación de alternativas de diseño

Durante el desarrollo de AuraBot se deben resolver decisiones que afectan la integración, el consumo de recursos y el mantenimiento.

3.1. PWM por hardware o por software

La Raspberry Pi 4 comparte recursos del periférico PWM entre la salida de audio analógica y los canales PWM disponibles para otros usos. Por ello, utilizar el mismo periférico para generar el audio y controlar la velocidad de los motores introduce un conflicto de configuración y temporización.

Se selecciona el PWM por software mediante una biblioteca propia que genera las transiciones de las señales utilizando GPIO y temporización con hilos. La implementación se separa en `libpwm` y `libgpio`, de manera que la generación de PWM no dependa directamente de los detalles de acceso al GPIO.

Esta alternativa permite conservar la salida de audio analógica y evita depender del periférico PWM de hardware para los motores. Como limitación, la temporización por software depende de la planificación del sistema operativo y debe validarse bajo carga.

3.2. Alternativas para la interfaz remota

Se utiliza una interfaz web en lugar de requerir una aplicación nativa instalada en cada dispositivo. La implementación del repositorio combina BusyBox HTTP, programas CGI en C y una interfaz construida con HTML, CSS y JavaScript.

Esta solución aprovecha herramientas disponibles en el entorno Linux y evita incorporar un servidor de aplicaciones más pesado. La interfaz permite consultar el estado del robot,

cambiar el modo de operación, controlar el movimiento y gestionar el audio.

3.3. Organización de la reproducción de audio

En lugar de ejecutar cada operación de reproducción directamente desde la interfaz web, AuraBot utiliza una biblioteca de audio y un servidor independiente. La comunicación con este servidor se realiza mediante la FIFO `/run/aurabot-audio/control`.

Esta separación permite mantener un único responsable de la reproducción y reduce la duplicación de código. La biblioteca `libaudio.so` controla `mpg123` y ALSA, mientras que el servidor procesa órdenes como reproducir, pausar, detener, ajustar volumen y seleccionar una pista.

4. DI3: Diseño de la solución seleccionada

4.1. Arquitectura del software

La arquitectura de AuraBot está organizada en los siguientes componentes:

1. **Interfaz web:** proporciona autenticación, controles de movimiento, consulta de sensores, mapa y funciones de audio.
2. **Biblioteca API de AuraBot:** expone operaciones de control a los clientes y se comunica con el proceso principal mediante IPC.
3. **Proceso `aurabot`:** coordina el estado del robot, la máquina de estados, los motores, los sensores, la odometría y la navegación.
4. **Bibliotecas de hardware:** separan las funciones de GPIO, PWM, audio, LEDs y sensores.
5. **Servidor de audio:** mantiene la reproducción activa y recibe órdenes mediante la FIFO.
6. **Sistema operativo:** ejecuta los procesos y administra los servicios mediante systemd.

El servidor web no escribe directamente sobre los pines de hardware. Las solicitudes pasan por la API y el proceso de control, lo cual centraliza las decisiones que afectan el movimiento.

4.2. Control y navegación autónoma

El módulo `autonomous.c` implementa la lógica de navegación reactiva. El controlador utiliza los sensores de proximidad para identificar obstáculos y coordina las maniobras de avance, detención, retroceso y giro.

La máquina de estados y los módulos de movimiento permiten organizar el comportamiento del robot y evitar que las solicitudes de control se procesen como acciones independientes sin considerar el estado actual.

Los encoders Hall se consultan para determinar si las ruedas continúan produciendo pulsos. El módulo de odometría utiliza esta información para estimar el desplazamiento, mientras que `map.c` mantiene la representación incremental del área explorada.

El modo manual se controla mediante solicitudes remotas. El sistema incorpora un mecanismo de concesión temporal del control y una parada de emergencia para evitar que múltiples clientes envíen órdenes de movimiento simultáneamente.

4.3. Audio y almacenamiento

La reproducción utiliza `mpg123` y ALSA a través de `libaudio.so`. El servidor acepta comandos mediante una FIFO y permite seleccionar pistas a partir de una lista de reproducción.

La partición de almacenamiento `AURA_AUDIO` se utiliza para conservar las canciones y la lista de reproducción bajo `/media/audio`. Los sonidos internos para eventos del robot se instalan por separado en la imagen.

Esta organización permite distinguir entre los archivos del usuario y los sonidos de notificación del sistema. Se contemplan eventos de inicio, activación del modo autónomo, cambio a modo manual y detección de obstáculos.

4.4. Interfaz web y autenticación

La interfaz web presenta el estado del robot y permite controlar el modo de operación, el movimiento, la reproducción y el volumen. También consulta los datos de los sensores y la información del mapa para actualizar la visualización.

El repositorio incluye autenticación mediante usuarios almacenados en SQLite, contraseñas protegidas mediante hashes bcrypt, roles, cookies de sesión y validación CSRF.

La interfaz utiliza HTTP sobre la red configurada. Esta solución requiere reconocer que HTTP no cifra por sí mismo las sesiones: la protección WPA2 de la red Wi-Fi no sustituye a TLS. El alcance de seguridad debe documentar esta limitación.

4.5. Construcción e integración con Yocto

El producto se integra mediante la capa `meta-ce1113`, que contiene recetas para las bibliotecas, las aplicaciones, los servicios y la configuración de hardware.

La imagen `ce1113-p1` se compone mediante grupos de paquetes separados por responsabilidad: bibliotecas, hardware y aplicaciones. CMake organiza la compilación de los componentes en C, mientras que BitBake gestiona su integración en la imagen final.

Los servicios principales se administran mediante systemd. Las unidades configuran el inicio automático y políticas de reinicio ante fallos, evitando depender de una sesión interactiva en la Raspberry Pi.

5. DI4: Validación del diseño

La validación debe distinguir entre la existencia de una implementación en el repositorio, las pruebas realizadas con hardware simulado y las verificaciones que requieren el robot físico.

Área	**Prueba propuesta**	**Evidencia requerida**
Navegación	Verificar la reacción ante obstáculos y los cambios de estado.	Registro de ejecución y prueba física.
Motores y PWM	Comprobar dirección, velocidad y parada de ambos motores.	Medición de señales y prueba con las ruedas levantadas.
Sensores y LEDs	Comparar las lecturas con obstáculos presentes y ausentes.	Estado reportado y fotografías del circuito.
Odometría y mapa	Comprobar la actualización de pulsos, desplazamiento y celdas.	Comparación con un recorrido controlado.
Audio	Reproducir, pausar, detener y cambiar volumen mientras opera el controlador.	Prueba de audio simultáneo con navegación.
Interfaz y seguridad	Verificar autenticación, permisos, concesión de control y parada de emergencia.	Resultados de pruebas HTTP y sesión web.
Yocto y servicios	Construir la imagen, comprobar el inicio automático y provocar un fallo controlado de un servicio.	Log de BitBake y registros de systemd.
Eficiencia	Medir rootfs, tiempo de arranque, RAM y CPU bajo carga.	Salida de las herramientas en la Raspberry Pi.

El repositorio contempla pruebas automatizadas con hardware simulado para componentes del núcleo, la máquina de estados, el mapa y la integración web. Estas pruebas permiten verificar la lógica sin depender de que todos los periféricos estén conectados.

Sin embargo, no sustituyen las mediciones de las señales físicas, la validación del aislamiento eléctrico ni las métricas obtenidas en la imagen final. Los resultados definitivos deben incorporarse a partir de las evidencias recogidas durante las pruebas en el equipo real.

6. Conclusiones

El diseño de AuraBot utiliza una arquitectura modular para separar el control del robot, la interfaz web, el acceso al hardware y la reproducción de audio. Esta organización facilita la depuración de los componentes y permite integrar el software en una imagen Linux construida mediante Yocto.

La decisión de utilizar PWM por software responde al conflicto entre el periférico PWM de la Raspberry Pi y la salida de audio analógica. A su vez, el uso de bibliotecas dinámicas y servicios independientes permite mantener responsabilidades claras entre los módulos.

La validación final requiere combinar las pruebas automatizadas con verificaciones en el hardware real, especialmente en lo relacionado con la seguridad eléctrica, la navegación, la reproducción simultánea de audio y las métricas de eficiencia.

Este documento fue generado con AI y posteriormente revisado y edita de forma manual